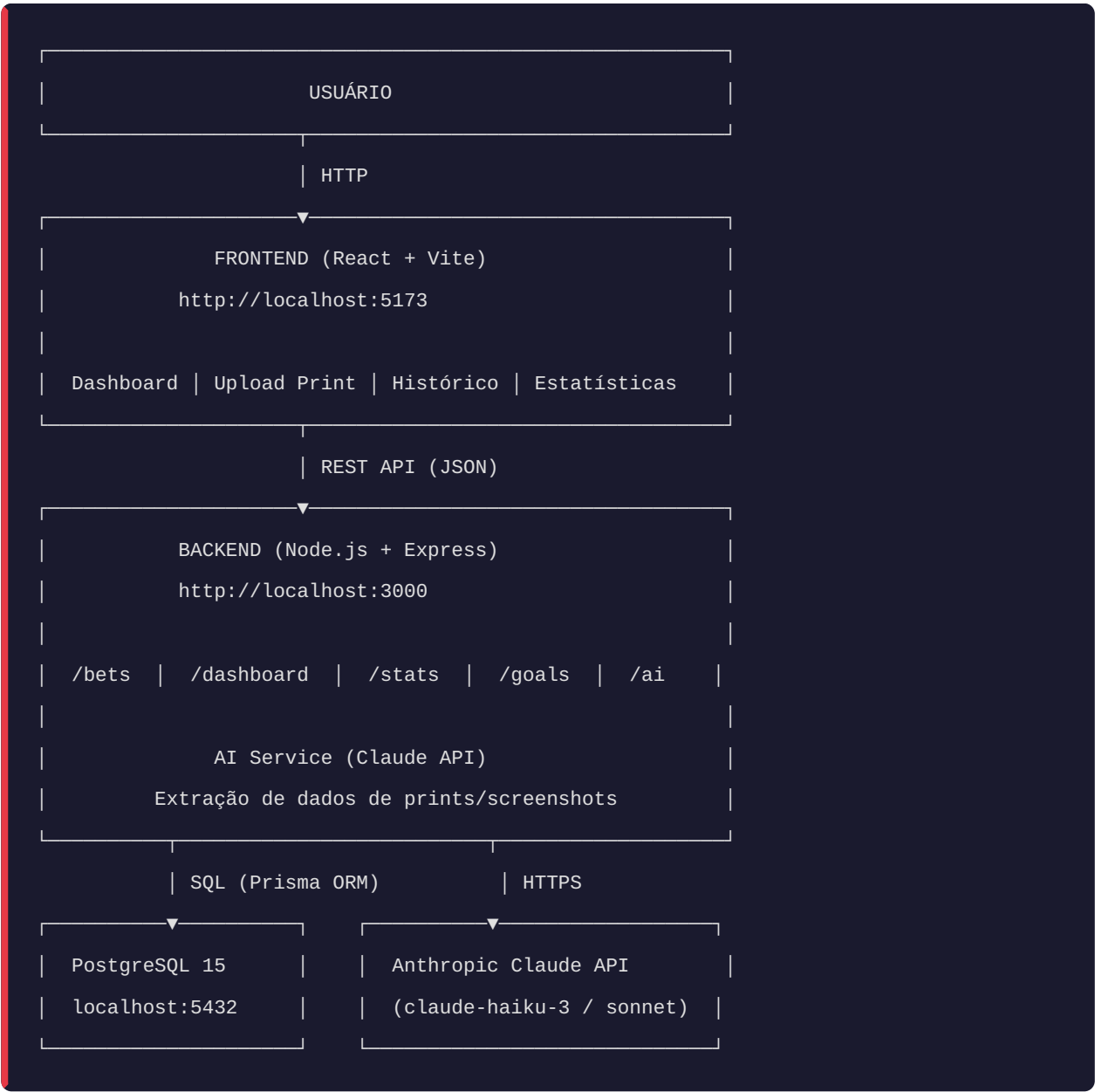


# Arquitetura Técnica — BetManager

## Visão Geral da Arquitetura

O BetManager segue uma arquitetura de **três camadas** (frontend, backend, banco de dados) com um módulo adicional de integração com IA. O sistema foi projetado para rodar localmente via Docker Compose, com capacidade de migração para nuvem sem alterações no código.



# Componentes

---

## Frontend — React + Vite + TypeScript

### Responsabilidades:

- Interface do usuário
- Upload de imagens para extração via IA
- Exibição de dashboard e gráficos
- Formulários de registro e edição de apostas

### Bibliotecas principais:

```
react 18          - UI
react-router-dom  - Roteamento
recharts          - Gráficos do dashboard
react-query       - Cache e estado de servidor
axios             - Chamadas HTTP
react-hook-form   - Formulários
react-dropzone    - Upload de arquivos
date-fns          - Manipulação de datas
tailwindcss       - Estilização
shadcn/ui         - Componentes de UI
```

### Estrutura de páginas:

```
/                → Dashboard principal
/apostas         → Histórico de apostas
/apostas/nova    → Registro manual ou via print
/estatisticas    → Análises por esporte, casa, tipo
/metas           → Configuração e acompanhamento de metas
```

---

## Backend — Node.js + Express + TypeScript

### Responsabilidades:

- API REST para todas as operações do sistema
- Integração com Claude API para extração de prints
- Cálculos de estatísticas e agregações
- Validação de dados com Zod

**Bibliotecas principais:**

```
express          - servidor HTTP
typescript       - tipagem estática
prisma           - ORM e migrations
@anthropic-ai/sdk - integração com Claude API
zod              - validação de schemas
multer           - upload de arquivos (prints)
jsonwebtoken     - JWT (fase 5)
```

**Estrutura dos routers:**

| Arquivo       | Prefixo        | Descrição                    |
|---------------|----------------|------------------------------|
| bets.ts       | /api/bets      | CRUD de apostas              |
| dashboard.ts  | /api/dashboard | Dados agregados do dashboard |
| statistics.ts | /api/stats     | Estatísticas por dimensão    |
| goals.ts      | /api/goals     | Metas mensais                |
| aiExtract.ts  | /api/ai        | Extração de dados via IA     |

**Banco de Dados — PostgreSQL 15**

**Justificativa do PostgreSQL sobre MySQL:**

- Window functions nativas (cálculo de lucro acumulado, médias móveis)
- CTEs recursivas para análises complexas
- Melhor suporte a tipos de dados (NUMERIC para valores monetários)
- Extensão `pg_stat_statements` para monitoramento de queries
- Migração mais simples para serviços cloud (Supabase, Railway, Neon)

**ORM e Migrations:**

- Prisma para acesso ao banco com tipagem automática
- Migrations geradas automaticamente pelo Prisma CLI

**Integração com IA — Claude API**

**Fluxo completo de extração:**

```
1. Frontend: usuário faz upload da imagem
    ↓
2. Backend recebe multipart/form-data
    ↓
3. aiService.ts converte imagem para base64
    ↓
4. Monta prompt estruturado:
    - System: "Você é um extrator de dados de apostas..."
    - User: [imagem em base64] + instrução de retorno JSON
    ↓
5. Claude API (claude-haiku-3 para custo, sonnet para accuracy)
    ↓
6. Response: JSON com array de apostas detectadas
    ↓
7. Backend valida com Zod + retorna ao frontend
    ↓
8. Frontend exibe para confirmação do usuário
    ↓
9. Usuário confirma → POST /api/bets (salva no banco)
```

#### **Prompt de extração (template):**

System:

Você é um especialista em extrair dados de apostas esportivas de screenshots. Retorne APENAS um JSON válido, sem markdown.

User:

Analise esta imagem de apostas e extraia todos os registros. Retorne no formato:

```
{
  "apostas": [
    {
      "data": "DD/MM",
      "descricao": "nome da aposta",
      "valor_apostado": 20.00,
      "casa": "Bet365|Superbet|...",
      "odd": 1.75,
      "retorno_total": 35.00,
      "resultado": "ganhou|perdeu|void|pendente"
    }
  ]
}
```

Para campos não identificáveis, use null.

### Modelo selecionado por caso:

- `claude-haiku-3` → extrações simples, prints limpos (menor custo)
- `claude-sonnet-4` → prints complexos, múltiplas colunas, baixa qualidade

## Docker Compose

---

```
version: '3.9'

services:
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    environment:
      - VITE_API_URL=http://localhost:3000

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    depends_on:
      - db
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - PORT=3000
    volumes:
      - ./backend:/app
      - /app/node_modules

  db:
    image: postgres:15-alpine
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_DB=${POSTGRES_DB}
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

```
volumes:
  postgres_data:
```

## Decisões de Arquitetura

### Por que Node.js + Express e não Python/FastAPI?

- Stack unificada em TypeScript (frontend e backend compartilham tipos)
- Ecossistema npm maduro com o SDK oficial da Anthropic ( [@anthropic-ai/sdk](#) )
- Prisma oferece DX superior ao SQLAlchemy com geração automática de tipos
- Familiaridade do desenvolvedor com JavaScript/TypeScript
- Performance suficiente para o volume de dados do projeto

### Por que não usar SQLite?

SQLite foi considerado por ser mais simples (sem Docker para o banco), mas descartado porque:

- Não suporta múltiplas conexões simultâneas (problema ao escalar)
- Window functions limitadas (necessárias para análises de lucro)
- Migração para nuvem exigiria reescrever queries

### Estratégia de migração para nuvem (Fase 5)

| Local (Docker)        | → | Nuvem                            |
|-----------------------|---|----------------------------------|
| PostgreSQL local      | → | Supabase (PostgreSQL gerenciado) |
| Backend local         | → | Railway ou Render                |
| Frontend local        | → | Vercel ou Netlify                |
| Variáveis de ambiente | → | Secrets do provedor              |

Nenhuma alteração de código — apenas mudança de variáveis de ambiente.

## Segurança (Fase Local)

- API sem autenticação na fase local (acesso restrito à rede local)
- Chave da Anthropic em variável de ambiente (nunca no `commit`)